

Algoritmos y Estructuras de Datos II

Laboratorio - 20/03/2025

Laboratorio 1: Ordenación

- Revisión 2026: Franco Luque / Marco Rocchietti

Código

lab01-kickstart.tar.gz

Objetivos

1. Introducir las cadenas en C
2. Repaso de **typedef**
3. Implementar el algoritmo de ordenación por inserción (*insertion-sort*)
4. Implementar el algoritmo de ordenación rápida (*quick-sort*)
5. Comparar desempeño de los algoritmos *selection-sort*, *insertion-sort* y *quick-sort* en distintos ejemplos
6. Lectura y comprensión del código entregado por la cátedra
7. Trabajar con implementaciones opacas de funciones leyendo su documentación
8. Abstraer la noción de orden
9. Usar procedimientos en C
10. Uso de funciones locales en módulos en C

Requerimientos

1. Compilar con los flags de la materia:

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 ...
```
2. Seguir las guías de estilo
3. Prohibido usar `break`, `continue` y `goto`!!
4. Prohibido usar `return` a la mitad de una función.

Recursos

Recursos generales:

- [Videos del Laboratorio en el aula virtual](#)
- [Documentación en el aula virtual](#)
 - **La biblia: El lenguaje de programación C.**
- Estilo de codificación:
 - [Guía de estilo para la programación en C](#)
 - [Consejos de Estilo de Programación en C](#)
- [Referencia de C](#) (cppreference)
- [The GNU C Library Reference Manual](#) (en inglés)

- Páginas del manual de linux: Comando “man 3 <función>”.
Ejemplo: “man 3 scanf”.
- Más:
 - [Referencia de C](#) (cppreference)
 - [The GNU C Library Reference Manual](#) (en inglés)
 - Páginas del manual de linux: Comando “man 3 <función>”.
Ejemplo: “man 3 scanf”.

Recursos específicos:

- Teóricos:
 - [Ordenación elemental](#)
 - [Ordenación avanzada](#)
- [Cadenas de caracteres](#)
- Tabla de caracteres ASCII (tipo char de C):
 - Comando “man ascii” (páginas del manual de linux)
- Módulos en C:
 -  Tutorial de C – 13. Archivos de cabecera y múltiples .c (parte 1)
- #ifndef y #define:
 - [¿Por qué se utilizan #ifndef y #define en los archivos de encabezado de C++?](#) (stackoverflow)
- Ruta absoluta vs relativa en Linux: ¿Cuál es la diferencia?
 - <https://itsfoss.com/es/ruta-absoluta-relativa-linux/>

Ejercicio 0: Cadenas de caracteres (“strings”)

Las cadenas en C se pueden pensar como arreglos de caracteres (más adelante veremos que también son punteros, pero por ahora... usamos arreglos). Los caracteres son valores del tipo `char` (que representa exactamente un carácter de 1 byte), entonces para guardar un *string* en C se puede usar el siguiente arreglo:

```
char cadena[5];
```

En este ejemplo el arreglo `cadena` tiene capacidad para guardar un *string* de hasta 4 (cuatro) caracteres de longitud. Esto es así porque toda cadena en C **debe terminar con el carácter '\0'** por lo cual ya tenemos un lugar ocupado. Esta convención permite saber dónde termina la cadena independientemente de la capacidad del arreglo. Entonces, se puede almacenar en `cadena` una palabra con longitud de entre uno y cuatro caracteres, pero incluso también se puede guardar una palabra vacía (en ese caso `cadena[0] = '\0'`). Si queremos armar el *string* con la palabra “hola” podemos hacer:

```
char cadena[5]={'h', 'o', 'l', 'a', '\0'};
printf("cadena: %s\n", cadena);
```

Es muy importante no olvidarse de poner el `'\0'` final ya que de lo contrario `printf()` va a recorrer a `cadena[]` por fuera de su rango hasta que aparezca un `'\0'` y al acceder a memoria inválida se producirá eventualmente una **violación de segmento** (*segmentation fault*). Notar que no hay ningún problema en hacer

```
char cadena[10]={'h', 'o', 'l', 'a', '\0'};
printf("cadena: %s\n", cadena);
```

ya que simplemente estamos usando cinco de los diez elementos del arreglo. Como el `'\0'` se encuentra en `cadena[4]`, la función `printf()` en el ejemplo anterior va a mostrar los caracteres que hay hasta esa posición (sin incluirla).

Otra forma más cómoda de armar el *string* con la palabra “hola” es hacer algo como:

```
char cadena[10]="hola";
printf("cadena: %s\n", cadena);
```

En este caso el caracter `'\0'` se agrega implícitamente en el arreglo `cadena`. Para no tener que contar la cantidad de caracteres que necesitamos se puede definir una cadena directamente haciendo:

```
char cadena[]="hola mundo!";
printf("cadena: %s\n", cadena);
```

el contenido del *array* es el siguiente:

cadena:	'h'	'o'	'l'	'a'	' '	'm'	'u'	'n'	'd'	'o'	'!'	'\0'
	0	1	2	3	4	5	6	7	8	9	10	11

Cabe destacar que todas estas maneras de armar *strings* solo son válidas al momento de inicialización del arreglo. Si hacemos

```
char cadena[]="hola mundo!";
cadena = "chau mundo!";
printf("cadena: %s\n", cadena);
```

El resultado va a ser un error de compilación. En este ejercicio vamos a trabajar con **cadenas guardadas en arreglos** que tienen un **tamaño fijo**. Vamos a definir un tipo para estas cadenas usando **typedef**. A modo de repaso, lo que hace **typedef** es dar un nombre nuevo a un tipo que ya existe en C. Entonces por ejemplo si queremos definir el tipo **natural** para los números positivos:

```
typedef unsigned int natural;
```

De esta manera cuando declaremos una variable del tipo **natural** la variable va a ser equivalente a una variable del tipo **unsigned int**. La sintaxis entonces es:

```
typedef <tipo-existente> <nuevo-tipo>;
```

Para definir el tipo nuevo **fixstring** hay que usar una sintaxis más extraña:

```
#define FIXSTRING_MAX 100

typedef char fixstring [FIXSTRING_MAX];
```

Lo que se hace aquí es definir el tipo **fixstring** como el tipo de los arreglos de elementos de tipo **char** que tienen una capacidad de **FIXSTRING_MAX** elementos. O sea que si declaramos

```
fixstring s;
```

es lo mismo que declarar

```
char s[FIXSTRING_MAX];
```

Dentro de la carpeta `ej0` van a encontrar los archivos `fixstring.h`, `fixstring.c` y `main.c`. Deben implementar en `fixstring.c` las funciones:

```
unsigned int fstring_length(fixstring s);

bool fstring_eq(fixstring s1, fixstring s2);

bool fstring_less_eq(fixstring s1, fixstring s2);
```

La función `fstring_length()` devuelve la longitud de la cadena guardada en el parámetro `s`, la función `fstring_eq()` indica si las cadenas `s1` y `s2` son iguales (contienen la misma cadena), mientras que `fstring_less_eq()` indica si la cadena guardada en `s1` es menor o igual que la guardada en `s2` en el sentido del orden alfabético. **No se permite usar librerías de C como `string.h` ni `strings.h`.** Una vez implementadas pueden probarlas compilando junto con `main.c`.

Compilación

Compilar ejecutando:

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 -c fixstring.c main.c
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 fixstring.o main.o -o main
```

Testing

En `tests.c` se incluyen casos de test para `fstring_eq()` y `fstring_less_eq()`.

Para compilar y ejecutar:

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 -c fixstring.c tests.c
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 fixstring.o tests.o -o tests
$ ./tests
```

Si alguno de los tests falla, el programa imprime los parámetros `s1` y `s2`, el resultado esperado (“expected”) y el resultado obtenido (“got”). Por ejemplo:

```
$ ./tests
TESTING fstring_eq
Test case 1: OK
Test case 2: OK
Test case 3: OK
Test case 4: OK
Test case 5: OK
Test case 6: OK
Test case 7: OK
Test case 8: OK

TESTING fstring_less_eq
```

```
Test case 1: OK
Test case 2: OK
Test case 3: OK
Test case 4: OK
Test case 5: OK
Test case 6: OK
Test case 7: OK
Test case 8: FAILED: fstring_less_eq("chau", "hola") expected 1 got 0.
```

Ejercicio 1: Insertion Sort

Dentro de la carpeta `ej1` se encuentran los siguientes archivos

Archivo	Descripción
<code>array_helpers.h</code>	Prototipos y descripciones de las funciones auxiliares para manipular arreglos.
<code>array_helpers.c</code>	Implementaciones de las funciones de la librería <code>array_helpers</code>
<code>sort_helpers.h</code>	Prototipos y descripciones de las funciones <code>goes_before()</code> , <code>swap()</code> y <code>array_is_sorted()</code>
<code>sort_helpers.c</code>	Implementaciones las funciones declaradas en <code>sort_helpers.h</code>
<code>sort.h</code>	Prototipo de la función <code>insertion_sort()</code> y su descripción
<code>sort.c</code>	Contiene una implementación incompleta de <code>insertion_sort()</code> , falta implementar <code>insert()</code>
<code>main.c</code>	Programa principal que carga un <i>array</i> de números, luego lo ordena con la función <code>insertion_sort()</code> y finalmente comprueba que el arreglo sea permutación ordenada del que se cargó inicialmente.

Parte A: Completado del Insertion Sort

Se debe realizar una implementación del algoritmo de ordenación por inserción (alias *insertion-sort*). Para ello es necesario completar la implementación del “procedimiento” `insert()` en el módulo `sort.c`. Como guía se puede examinar el resto del archivo `sort.c` y la definición del [algoritmo de ordenación por inserción vista en el teórico](#). El algoritmo debe ordenar con respecto a la relación `goes_before()` declarada en `sort_helpers.h` cuya implementación puede verse en `sort_helpers.c`.

Ayudas:

- Realizar este ejercicio conjuntamente con la parte B para poder detectar errores.
- Adaptar el algoritmo para arreglos indexados desde 0 (en el teórico se indexan desde 1).
- Adaptar el algoritmo del teórico para usar `goes_before()`.
- Compilar y testear usando las instrucciones de compilación indicadas más abajo.
- **No usar flag `-Werror` para compilar hasta el final.**

Parte B: Impresión de pasos intermedios

Imprimir el arreglo completo en cada modificación que se realice durante el proceso de ordenación. Para esto usar `array_dump()` de `array_helpers`.

Crear un archivo de entrada de ejemplo (como `input/example-unsorted.in`) con [un arreglo idéntico al mostrado en el teórico](#):

```
7
9 3 1 3 5 2 7
```

Verificar que lo que se imprime se corresponde con los pasos mostrados en el teórico:

9	3	1	3	5	2	7	1	3	3	5	2	9	7
9	3	1	3	5	2	7	1	3	3	2	5	9	7
3	9	1	3	5	2	7	1	3	2	3	5	9	7
3	1	9	3	5	2	7	1	2	3	3	5	9	7
1	3	9	3	5	2	7	1	2	3	3	5	7	9
1	3	3	9	5	2	7							
1	3	3	5	9	2	7							

Parte C: Chequeo de Invariante

Se debe modificar el procedimiento `insertion_sort()` agregando la verificación de cumplimiento de la invariante del ciclo `for` que se vio en el teórico. Por simplicidad sólo se debe verificar la siguiente parte de la Invariante:

- el segmento inicial `a[0,i)` del arreglo está ordenado.

Para ello usar las funciones `assert()` y `array_is_sorted()`.

Compilación

Para todos los incisos se puede compilar ejecutando:

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 -c array_helpers.c sort.c sort_helpers.c main.c
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 array_helpers.o sort.o sort_helpers.o main.o -o sorter
```

El programa puede ejecutarse de la siguiente manera:

```
$ ./sorter ../input/example-unsorted.in
```

Si el programa funciona bien en ese ejemplo (es decir, si no reporta error), probar con otros archivos de la carpeta `../input`, sin olvidar realizar una prueba con el archivo `../input/empty.in`

Ejercicio 2: Quick Sort I

En este ejercicio se realizará una implementación *top-down* del algoritmo de ordenación rápida vista en el teórico. En la carpeta `ej2` se encuentran los siguientes archivos:

Archivo*	Descripción
<code>array_helpers.h</code>	Es el mismo que en el ejercicio anterior.
<code>array_helpers.c</code>	Es el mismo que en el ejercicio anterior.
<code>sort_helpers.h</code>	Contiene además la declaración y descripción de <code>partition()</code>
<code>sort_helpers.o</code>	Contiene implementaciones ilegibles de esas funciones (código compilado para la arquitectura x86-64)
<code>sort.h</code>	Contiene descripción de la función <code>quick_sort()</code>
<code>sort.c</code>	Contiene una implementación muy incompleta de <code>quick_sort()</code> , además falta implementar <code>quick_sort_rec()</code>
<code>main.c</code>	Contiene el programa principal que carga un arreglo de números, luego lo ordena con la función <code>quick_sort()</code> y finalmente comprueba que el arreglo sea una permutación ordenada del que se cargó inicialmente.

Parte A: Implementación de `quick_sort_rec()`

Implementar el “procedimiento” `quick_sort_rec()` en el archivo `sort.c`. Tener en cuenta que **no es necesario** implementar la función `partition()` puesto que la misma ya está implementada (aunque no puede leerse su código por estar compilada en `sort_helpers.o`). Para saber cómo utilizarla, examinar su descripción en `sort_helpers.h`.

A modo de guía se puede revisar la presentación del algoritmo de ordenación rápida realizada en la [clase del teórico](#).

Ayudas:

- Realizar este ejercicio conjuntamente con la parte B para poder detectar errores.
- Leer y entender la parte teórica del Quick Sort.
- Leer con atención la documentación de la función `partition()` en `sort_helpers.h`.
- Probar primero con arreglos de tamaño 1 y 2. Después ir agregando elementos de a uno y testeando.

Parte B: Función `main()`

Se debe abrir el archivo `main.c` y completar la función `main()` con una llamada al “procedimiento” `quick_sort()`. Para entender cómo utilizar este “procedimiento”, examinar el archivo `sort.h`.

Compilación

Compilar ejecutando:

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 -c array_helpers.c sort.c main.c
```

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 -no-pie array_helpers.o sort.o sort_helpers.o main.o -o sorter
```

La opción **-no-pie** tiene que ver con que se está usando un objeto precompilado `sort_helpers.o`, cuya compilación fue realizada en una computadora distinta. Esta opción puede ser necesaria para lograr compatibilidad entre los archivos binarios durante el “linkeo” y así poder generar el ejecutable.

Ejercicio 3: Quick Sort II: Partition

En la carpeta `ej3` se encuentran los siguientes archivos

Archivo	Descripción
<code>sort_helpers.h</code>	Contiene descripciones de las funciones <code>goes_before()</code> , <code>swap()</code> y <code>array_is_sorted()</code>
<code>sort_helpers.o</code>	Contiene implementaciones ilegibles de todo lo descrito en <code>sort_helpers.h</code> (código compilado para la arquitectura x86-64). Notar que la función <code>partition()</code> no está más aquí.
<code>sort.h</code>	Contiene descripción de la función <code>quick_sort()</code>
<code>sort.c</code>	contiene una implementación incompleta de <code>quick_sort()</code> , falta implementar <code>quick_sort_rec()</code> y <code>partition()</code> .

Copiar los archivos `array_helpers.h`, `array_helpers.c` y `main.c` del *ejercicio 2*. Luego copiar el “procedimiento” `quick_sort_rec()` (también del *ejercicio 2*) en el archivo `sort.c` y **definir** allí la función `partition()` usando como guía la presentación que se dio del algoritmo de ordenación rápida en la [clase del teórico](#).

Ayudas:

- Adaptar el algoritmo del teórico para usar `goes_before()`.
- **Imprimir el arreglo antes y después** de todas las llamadas a `partition`, así como también los parámetros de entrada izq, der y el resultado ppiv.
Verificar que la función se comporta como debería.
- Usar también las ayudas del ejercicio anterior.

Compilación

Una vez completada la definición de `partition()`, compilar el código con `gcc` siguiendo el mismo método del ejercicio 2.

Ejercicio 4: Versus

Realizar una comparación de todos los algoritmos de ordenación implementados en este laboratorio. En la carpeta **ej4** se encuentran los siguientes archivos:

Archivo	Descripción
sort_helpers.h	Se agregan nuevas declaraciones de funciones para manejo de contadores
sort_helpers.o	Contiene implementaciones ilegibles de todo lo descrito en sort_helpers.h (código compilado para la arquitectura x86-64)
sort.h	Contiene las declaraciones y descripciones de las implementaciones de los métodos de ordenación <i>selection-sort</i> , <i>insertion-sort</i> y <i>quick-sort</i>
sort.c	Contiene las definiciones incompletas de las funciones declaradas en sort.h . Deben completarse con el código de los ejercicios anteriores.
main.c	Contiene el programa principal que carga un arreglo de números, luego lo ordena usando alguno de los algoritmos de ordenación implementados y muestra: • Tiempo de ejecución • Número de comparaciones • Intercambios realizados.

Copiar los archivos **array_helpers.h** y **array_helpers.c** del ejercicio anterior y luego:

1. Abrir el archivo **sort.c** y copiar el código de cada uno de los algoritmos de ordenación resueltos en los ejercicios anteriores.
2. Abrir el archivo **main.c** y completar la función `main()` siguiendo los pasos indicados en los comentarios.

Compilación y Ejecución

Una vez completados los ítems 1 y 2, compilar el código con **gcc** siguiendo el mismo método de los ejercicios anteriores.

Analizar los resultados de la ejecución del programa para distintos ejemplos y sacar conclusiones sobre el desempeño de cada algoritmo de ordenación.

Ejercicio 5: Ordenación alfabética

En la carpeta **ej5** van a encontrar los archivos **fixstring.h**, **fixstring.c**. Deben copiar las implementaciones de **fstring_length()**, **fstring_eq()** y **fstring_less_eq()** realizadas en el *ejercicio 0* a **fixstring.c** y luego completar la implementación de las función

```
void fstring_swap(fixstring s1, fixstring s2);
```

que debe intercambiar los contenidos de las cadenas **s1** y **s2**. Para implementar esta función pueden utilizar de manera auxiliar a **fstring_set()** (que ya viene implementada).

Se incluye un código muy parecido al de los ejercicios anteriores pero que es capaz de leer un arreglo de palabras, es decir un arreglo de elementos de tipo **fixstring**. Los archivos de entrada están en la carpeta **ej5/input**. La idea será ordenar las palabras usando el algoritmo de *quick sort*. Deben entonces adaptar el código para que:

- a) Se ordene el arreglo de *strings* de entrada de manera alfabética
- b) Se ordene el arreglo de *strings* de entrada según el largo de las cadenas (las cadenas más cortas al principio).

La forma de compilar el código es:

```
$ gcc -Wall -Werror -Wextra -pedantic -std=c99 -c array_helpers.c fixstring.c main.c sort.c sort_helpers.c  
$ gcc -Wall -Wextra -std=c99 array_helpers.o fixstring.o main.o sort.o sort_helpers.o -o word_sorter
```

Y luego un ejemplo de ejecución:

```
$ ./word_sorter input/example-easywords.in  
4  
casa chau hola perro
```

Tener en cuenta que para fijar el criterio de ordenación van a necesitar modificar uno de los módulos. Incluyan una versión de dicho módulo que ordene como se pide en el punto a) y otra versión con el criterio del punto b).